
# Assignment
# on
# Spam Email Detection Using Naive Bayes Classifier

Course: CSE 411

Course Title: Data Science

Session: 2021-2022

**Submitted By**

**Md Ashraful Islam**
Roll: 22102001
Reg: 10807

**Supervised By**

**Prof. Dr. Tushar Kanti Saha**
Department of CSE
Jatiya Kabi Kazi Nazrul Islam University

**Jatiya Kabi Kazi Nazrul Islam University**
Department of Computer Science and Engineering
Trishal, Mymensingh

April 28, 2026

# Contents

# Chapter 1

# Introduction

## 1.1 Overview

Email is one of the most widely used communication methods for academic, professional, and personal purposes. However, a large number of unwanted and harmful emails are sent every day. These unwanted emails are generally known as spam emails. Spam emails may contain advertisements, fake offers, fraud links, lottery messages, phishing attempts, or harmful attachments.

The objective of this assignment is to develop a machine learning-based spam email detection system. The system checks whether a given email is spam or not using a supervised machine learning approach. According to the assignment instruction, the model is developed using the Naive Bayes classifier and the provided spam email dataset.

## 1.2 Problem Statement

Manual identification of spam emails is time-consuming and inefficient. A user may receive many emails every day, and it is difficult to manually check each email. Therefore, an automated system is required to classify emails as spam or not spam. The main problem of this assignment is to design and implement a machine learning model that can learn from labeled email data and predict the class of new email messages.

## 1.3 Objectives

The major objectives of this assignment are:

- To load and analyze the provided spam email dataset.
- To apply Exploratory Data Analysis (EDA) on the dataset.
- To preprocess email text data for machine learning.
- To convert text into numerical features using TF-IDF vectorization.

- To train a Naive Bayes classifier using an 80:20 train-test split.
- To evaluate the model using accuracy, classification report, and confusion matrix.
- To test the trained model with new email messages.

## 1.4 Assignment Requirement

The assignment instruction was to check whether an email is spam or not using the Naive Bayes classifier and the provided dataset. In addition, EDA and a machine learning model were required. This report therefore includes dataset analysis, methodology, implementation code, output, model explanation, and result analysis.

# Chapter 2

# Dataset and Exploratory Data Analysis

## 2.1 Dataset Description

The dataset used in this assignment is named `emails.csv`. It contains email text and the corresponding class label. The dataset has two columns: `text` and `spam`. The `text` column contains the email message, while the `spam` column represents the target label.

Table 2.1: Dataset Column Description

| **Column Name** | **Data Type** | **Description** |
|---|---|---|
| text | String | Contains the complete email message. |
| spam | Integer | Target class label where 1 means spam and 0 means not spam. |

## 2.2 Dataset Summary

The dataset contains a total of 5,728 email records. Among them, 4,360 emails are not spam and 1,368 emails are spam. This shows that the dataset is imbalanced, because the number of non-spam emails is higher than the number of spam emails.

Table 2.2: Dataset Summary

| **Property** | **Value** |
|---|---|
| Dataset File | emails.csv |
| Total Records | 5728 |
| Total Columns | 2 |
| Text Column | text |
| Label Column | spam |
| Missing Values | 0 |

## 2.3 Class Distribution

Table 2.3: Class Distribution of Emails

| Class Label | Meaning | Number of Emails |
|---|---|---|
| 0 | Not Spam | 4360 |
| 1 | Spam | 1368 |

The class distribution shows that most of the emails in the dataset are not spam. Although the dataset is not perfectly balanced, it contains enough spam and non-spam examples for training a supervised classification model.

## 2.4 EDA Analysis

Exploratory Data Analysis was performed to understand the structure and quality of the dataset. The EDA process included checking the dataset shape, column names, missing values, label distribution, email length, and word count distribution.

Table 2.4: Email Length and Word Count Statistics

| Statistic | Email Length | Word Count |
|---|---|---|
| Count | 5728 | 5728 |
| Mean | 1556.77 | 326.85 |
| Standard Deviation | 2042.65 | 418.78 |
| Minimum | 13 | 2 |
| 25% | 508.75 | 101 |
| 50% | 979 | 210 |
| 75% | 1894.25 | 401 |
| Maximum | 43952 | 8477 |

The EDA result shows that email messages vary significantly in length. Some emails are very short, while some are extremely long. This variation is important because spam emails often contain promotional text, repeated phrases, or long advertising content. On the other hand, non-spam emails may contain academic or official messages with different writing styles.

## 2.5 EDA Figures

The following figures can be inserted after generating them from the Python code. Place the generated images inside a folder named `figures`.

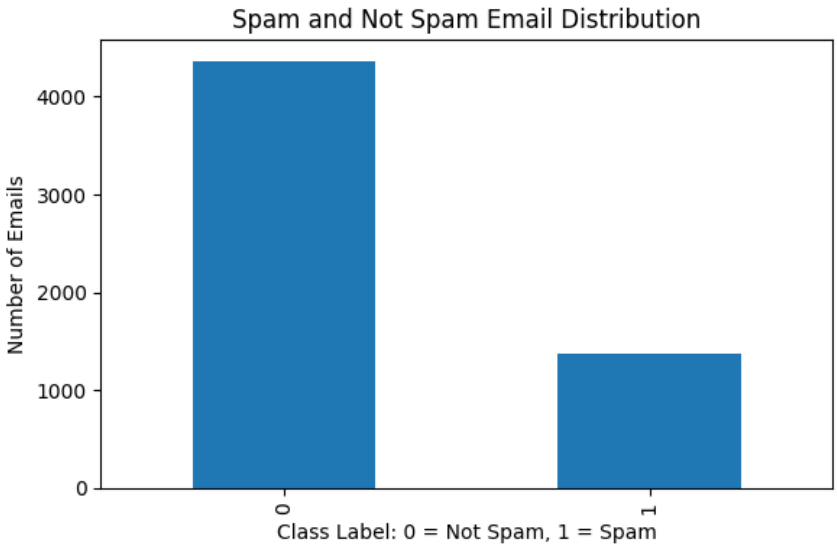


Figure 2.1: Spam and Not Spam Email Distribution

Figure 2.1 shows the distribution of spam and non-spam emails. The chart indicates that non-spam emails are more frequent than spam emails in the dataset.

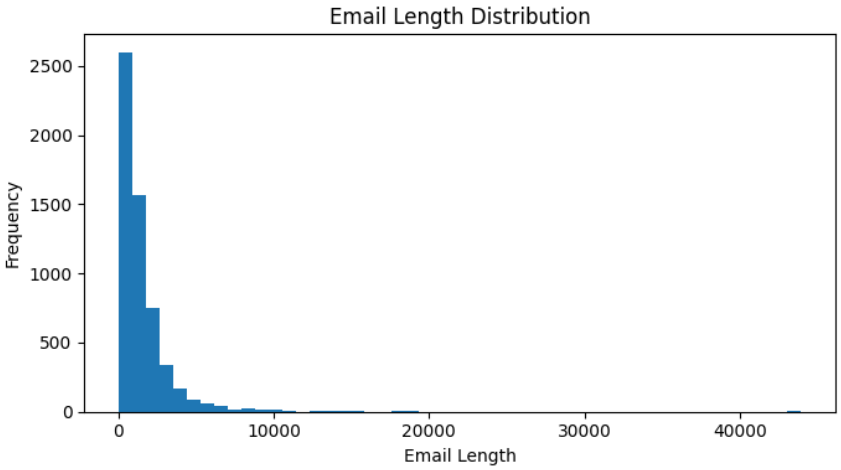


Figure 2.2: Email Length Distribution

Figure 2.2 shows the distribution of email length. This helps to understand how long or short the email messages are.

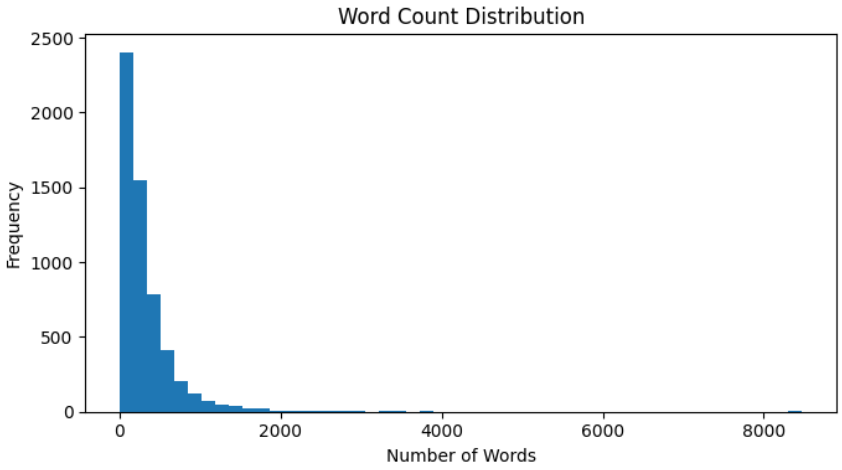


Figure 2.3: Word Count Distribution

Figure 2.3 shows the word count distribution. Word count is useful for analyzing the text structure of emails.

# Chapter 3

# Methodology

## 3.1 System Workflow

The spam email detection system follows a structured machine learning workflow. First, the dataset is loaded. Then EDA is applied to understand the data. After that, the text data is converted into numerical features using TF-IDF. Finally, the Naive Bayes classifier is trained and evaluated.

Table 3.1: Machine Learning Workflow

| **Step** | **Process** | **Description** |
|---|---|---|
| 1 | Dataset Loading | The `emails.csv` file is loaded using pandas. |
| 2 | EDA | Dataset shape, missing values, class distribution, email length, and word count are analyzed. |
| 3 | Data Cleaning | Missing values are removed and labels are converted into integer format. |
| 4 | Train-Test Split | The dataset is divided into 80% training data and 20% testing data. |
| 5 | Feature Extraction | TF-IDF is used to convert email text into numerical feature vectors. |
| 6 | Model Training | Multinomial Naive Bayes classifier is trained using training data. |
| 7 | Model Evaluation | Accuracy, classification report, and confusion matrix are generated. |
| 8 | Prediction | New email text is classified as spam or not spam. |

## 3.2 Train-Test Split

The dataset was divided into 80% training data and 20% testing data. The training data was used to teach the model, while the testing data was used to evaluate the performance of the trained model.

Table 3.2: Train-Test Split

| **Data Part** | **Number of Records** |
|---|---|
| Training Data | 4582 |
| Testing Data | 1146 |
| Total Data | 5728 |

## 3.3 TF-IDF Vectorization

Machine learning models cannot directly understand raw text. Therefore, the email text must be converted into numerical form. In this project, TF-IDF vectorization was used. TF-IDF stands for Term Frequency-Inverse Document Frequency. It gives importance to words that are frequent in a particular email but not common in all emails.

The TF-IDF value is calculated as:

$$TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$$

where,

$$IDF(t) = \log\left(\frac{N}{DF(t)}\right)$$

Here, $TF(t,d)$ is the frequency of term $t$ in document $d$, $N$ is the total number of documents, and $DF(t)$ is the number of documents containing term $t$.

## 3.4 Naive Bayes Classifier

Naive Bayes is a probabilistic machine learning algorithm based on Bayes' theorem. It is widely used for text classification problems such as spam detection because it performs well with word-frequency based features.

Bayes' theorem is written as:

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

where,

- $P(C|X)$ is the posterior probability of class $C$ given input $X$.
- $P(X|C)$ is the likelihood of input $X$ given class $C$.
- $P(C)$ is the prior probability of class $C$.
- $P(X)$ is the evidence or probability of input $X$.

For spam email detection, the model compares the probability of two classes:

$$P(Spam|Email)$$

and

$$P(NotSpam|Email)$$

The final class is selected using the following decision rule:

$$\hat{C} = \arg\max_{C} P(C) \prod_{i=1}^{n} P(x_i|C)$$

Here, $x_i$ represents each word or feature in the email. The model selects the class that has the highest posterior probability.

## 3.5 Multinomial Naive Bayes

In this assignment, Multinomial Naive Bayes was used because the input features are based on word frequency and TF-IDF values. It is suitable for text classification tasks where the features represent word occurrence or importance.

To avoid zero probability for unseen words, smoothing is applied. The probability of a word given a class is calculated as:

$$P(x_i|C) = \frac{count(x_i, C) + \alpha}{\sum_{j=1}^{n} count(x_j, C) + \alpha n}$$

where $\alpha$ is the smoothing parameter. In this project, $\alpha = 0.1$ was used.

## 3.6 Methodology Summary

Table 3.3: Methodology Summary

| **Component** | **Used Method** |
|---|---|
| Programming Language | Python |
| Dataset | emails.csv |
| EDA Library | pandas, matplotlib |
| Feature Extraction | TF-IDF Vectorizer |
| Machine Learning Model | Multinomial Naive Bayes |
| Training Data | 80% |
| Testing Data | 20% |
| Evaluation Metrics | Accuracy, Precision, Recall, F1-score, Confusion Matrix |
| Model Saving Tool | joblib |

# Chapter 4

# Implementation

## 4.1 Development Environment

The project was implemented using Python in Visual Studio Code. A virtual environment was used to manage the required Python packages.

Table 4.1: Development Environment

| **Tool/Technology** | **Purpose** |
|---|---|
| Python 3 | Programming language |
| VS Code | Code editor |
| pandas | Dataset loading and analysis |
| matplotlib | EDA visualization |
| scikit-learn | Machine learning model development |
| joblib | Saving and loading trained model |

## 4.2 Project Directory Structure

```
Spam Email Detector/
|
|-- emails.csv
|-- train_model.py
|-- predict.py
|-- requirements.txt
|-- spam_email_detector.pkl
|-- figures/
|   |-- label_distribution.png
|   |-- email_length_distribution.png
|   |-- word_count_distribution.png
|   |-- confusion_matrix.png
|-- venv/
```

## 4.3 Training Code

```
import os
import pandas as pd
import joblib
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score,
   classification_report, confusion_matrix


DATASET_PATH = "emails.csv"
TEXT_COL = "text"
LABEL_COL = "spam"
FIGURE_DIR = "figures"


def create_directory():
    if not os.path.exists(FIGURE_DIR):
        os.makedirs(FIGURE_DIR)


def perform_eda(df):
    print("\n========== Exploratory Data Analysis ==========")

    print("\nDataset Shape:")
    print(df.shape)

    print("\nDataset Columns:")
    print(df.columns.tolist())

    print("\nFirst 5 Rows:")
    print(df.head())

    print("\nMissing Values:")
    print(df.isnull().sum())
```

```
    print("\nLabel Distribution:")
    print(df[LABEL_COL].value_counts())

    df["email_length"] = df[TEXT_COL].astype(str).apply(len)
    df["word_count"] = df[TEXT_COL].astype(str).apply(lambda x:
        len(x.split()))

    print("\nEmail Length Statistics:")
    print(df["email_length"].describe())

    print("\nWord Count Statistics:")
    print(df["word_count"].describe())

    plt.figure(figsize=(6, 4))
    df[LABEL_COL].value_counts().plot(kind="bar")
    plt.title("Spam and Not Spam Email Distribution")
    plt.xlabel("Class Label: 0 = Not Spam, 1 = Spam")
    plt.ylabel("Number of Emails")
    plt.tight_layout()
    plt.savefig(f"{FIGURE_DIR}/label_distribution.png")
    plt.close()

    plt.figure(figsize=(7, 4))
    plt.hist(df["email_length"], bins=50)
    plt.title("Email Length Distribution")
    plt.xlabel("Email Length")
    plt.ylabel("Frequency")
    plt.tight_layout()
    plt.savefig(f"{FIGURE_DIR}/email_length_distribution.png")
    plt.close()

    plt.figure(figsize=(7, 4))
    plt.hist(df["word_count"], bins=50)
    plt.title("Word Count Distribution")
    plt.xlabel("Number of Words")
    plt.ylabel("Frequency")
    plt.tight_layout()
    plt.savefig(f"{FIGURE_DIR}/word_count_distribution.png")
    plt.close()

    return df
```

```
def train_model(df):
    df = df[[TEXT_COL, LABEL_COL]].dropna()

    X = df[TEXT_COL].astype(str)
    y = df[LABEL_COL].astype(int)

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42,
        stratify=y
    )

    print("\n========== Train-Test Split ==========")
    print("Training data size:", len(X_train))
    print("Testing data size:", len(X_test))

    model = Pipeline([
        ("tfidf", TfidfVectorizer(
            stop_words="english",
            lowercase=True,
            max_features=10000,
            ngram_range=(1, 2)
        )),
        ("classifier", MultinomialNB(alpha=0.1))
    ])

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)

    print("\n========== Model Evaluation ==========")
    print("Model Used: Multinomial Naive Bayes")
    print("Accuracy:", accuracy)
```

```
    print("\nClassification Report:")
    print(classification_report(
        y_test,
        y_pred,
        target_names=["Not Spam", "Spam"]
    ))

    print("\nConfusion Matrix:")
    print(cm)

    plt.figure(figsize=(5, 4))
    plt.imshow(cm)
    plt.title("Confusion Matrix")
    plt.xlabel("Predicted Label")
    plt.ylabel("Actual Label")
    plt.xticks([0, 1], ["Not Spam", "Spam"])
    plt.yticks([0, 1], ["Not Spam", "Spam"])

    for i in range(2):
        for j in range(2):
            plt.text(j, i, cm[i, j], ha="center", va="center")

    plt.tight_layout()
    plt.savefig(f"{FIGURE_DIR}/confusion_matrix.png")
    plt.close()

    joblib.dump(model, "spam_email_detector.pkl")

    print("\nModel saved successfully as spam_email_detector.
       pkl")

    return accuracy


def main():
    create_directory()

    df = pd.read_csv(DATASET_PATH)

    df = perform_eda(df)
```

```
    accuracy = train_model(df)

    print("\n========== Final Summary ==========")
    print("Dataset:", DATASET_PATH)
    print("Classifier: Multinomial Naive Bayes")
    print("Train-Test Split: 80% training, 20% testing")
    print(f"Final Accuracy: {accuracy * 100:.2f}%")
    print("EDA charts saved inside the figures folder.")


if __name__ == "__main__":
    main()
```

Listing 4.1: Training Code for Spam Email Detection

## 4.4 Prediction Code

```
import joblib


MODEL_PATH = "spam_email_detector.pkl"


def predict_email(email_text, model):
    prediction = model.predict([email_text])[0]
    probability = model.predict_proba([email_text])[0]

    not_spam_probability = probability[0] * 100
    spam_probability = probability[1] * 100

    if prediction == 1:
        result = "Spam Email"
    else:
        result = "Not Spam / Ham Email"

    return result, spam_probability, not_spam_probability


def main():
    model = joblib.load(MODEL_PATH)
```

```
    print("Spam Email Detector")
    print("Type 'exit' to stop.\n")

    while True:
        email_text = input("Enter email text: ")

        if email_text.lower() == "exit":
            print("Program closed.")
            break

        result, spam_prob, ham_prob = predict_email(email_text,
            model)

        print("\nPrediction:", result)
        print(f"Spam probability: {spam_prob:.2f}%")
        print(f"Not spam probability: {ham_prob:.2f}%\n")


if __name__ == "__main__":
    main()
```

Listing 4.2: Prediction Code for New Email Input

# Chapter 5

# Output and Results

## 5.1 Training Output

The model was trained successfully using the provided dataset. The following table summarizes the main training output.

Table 5.1: Training Output Summary

| **Output Item** | **Value** |
|---|---|
| Dataset Shape | 5728 rows, 2 columns |
| Dataset Columns | text, spam |
| Not Spam Emails | 4360 |
| Spam Emails | 1368 |
| Training Data Size | 4582 |
| Testing Data Size | 1146 |
| Classifier | Multinomial Naive Bayes |
| Accuracy | 99.04% |
| Saved Model | spam_email_detector.pkl |

## 5.2 Classification Report

Table 5.2: Classification Report

| **Class** | **Precision** | **Recall** | **F1-score** | **Support** |
|---|---|---|---|---|
| Not Spam | 0.99 | 0.99 | 0.99 | 872 |
| Spam | 0.98 | 0.98 | 0.98 | 274 |
| Accuracy | 0.99 | | | 1146 |
| Macro Average | 0.99 | 0.99 | 0.99 | 1146 |
| Weighted Average | 0.99 | 0.99 | 0.99 | 1146 |

## 5.3 Confusion Matrix

Table 5.3: Confusion Matrix

| Actual / Predicted | Not Spam | Spam |
|---|---|---|
| Not Spam | 866 | 6 |
| Spam | 5 | 269 |

The confusion matrix shows that the model correctly classified 866 non-spam emails and 269 spam emails. Only 6 non-spam emails were incorrectly classified as spam, and only 5 spam emails were incorrectly classified as not spam. Therefore, the total number of misclassified emails was 11 out of 1146 testing samples.

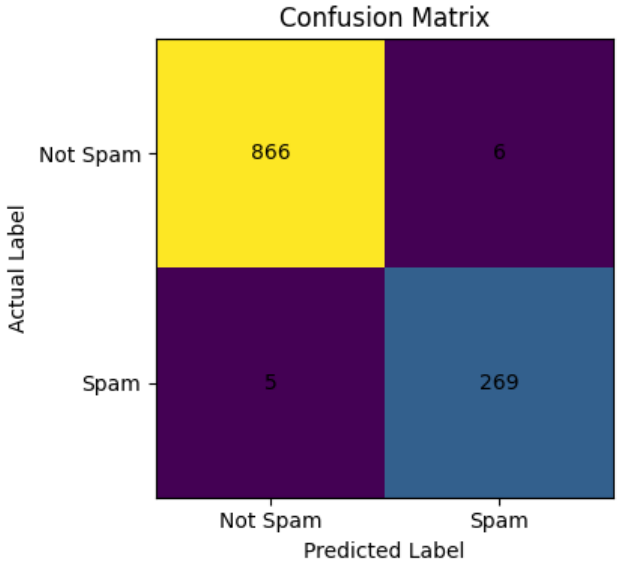


Figure 5.1: Confusion Matrix of Naive Bayes Classifier

## 5.4 Sample Prediction Output

The trained model was tested with new email messages. Promotional and reward-based messages were classified as spam, while academic and meeting-related messages were classified as not spam.

Table 5.4: Sample Prediction Results

| **Input Email Text** | **Prediction** |
|---|---|
| Congratulations! You have won a free iPhone. Click the link now to claim your prize. | Spam Email |
| Sir, I have completed the assignment and submitted it before the deadline. | Not Spam / Ham Email |
| Urgent! Your account has been selected for a $1000 cash reward. Click here to claim now. | Spam Email |
| Dear student, your class test will be held tomorrow at 10 AM. Please be present on time. | Not Spam / Ham Email |
| Your meeting with the supervisor has been scheduled for Sunday at 2 PM. | Not Spam / Ham Email |

## 5.5 Result Analysis

The final accuracy of the model is 99.04%, which indicates strong performance on the testing dataset. The precision, recall, and F1-score values are also high for both classes. This means the model is able to detect both spam and non-spam emails effectively.

The spam class achieved 0.98 precision and 0.98 recall. This indicates that the model can identify most spam emails correctly and produces very few false spam predictions. The non-spam class also achieved strong performance, which is important because wrongly marking important emails as spam can create practical problems for users.

Overall, the result proves that TF-IDF vectorization combined with Multinomial Naive Bayes is effective for spam email detection.

# Chapter 6

# Conclusion

## 6.1 Concluding Remarks

This assignment successfully developed a spam email detection system using the Naive Bayes classifier. The provided dataset was analyzed using EDA, and the text data was converted into numerical features using TF-IDF vectorization. The dataset was divided into 80% training data and 20% testing data. After training, the Multinomial Naive Bayes model achieved 99.04% accuracy on the testing dataset.

## 6.2 Limitations

Although the model performed well, there are some limitations:

- The model was trained on a fixed dataset, so performance may vary on a completely different dataset.
- The system only classifies text-based emails and does not analyze attachments.
- The current implementation runs in a command-line interface.
- The model may not detect advanced phishing emails that use very natural language.

## 6.3 Future Scopes

The project can be improved in the future in the following ways:

- A web-based interface can be developed for easier user interaction.
- More datasets can be added to improve generalization.
- Deep learning models such as LSTM or transformer-based models can be tested.
- Email attachments and URLs can be analyzed for better spam and phishing detection.